

Strings in C++ From Characters to Pattern Matching

A complete, from-zero guide to strings for on-campus placement interviews: what a character really is, everything `std::string` can do, frequency counting, two pointers, the universal sliding-window template, palindromes, parsing, KMP, rolling hashes, string DP, the 14 patterns, the bug catalogue and a curated 38-problem list.

113

SECTIONS

14

PATTERNS

91

CODE BLOCKS

38

PROBLEMS

Prepared for **Hari** · on-campus placement preparation
Guide + a companion solved-questions book of all 38 problems
Generated 15 September 2026

Contents

Part 0 · Strings from zero

- 0.1 A character is a number
- 0.2 The classification functions
- 0.3 C strings versus `std::string`
- 0.4 Small String Optimisation
- 0.5 Strings are mutable, and that matters
- 0.6 Reading input without losing your mind

Part 1 · The `std::string` toolkit

- 1.1 Creating and sizing
- 1.2 Accessing characters
- 1.3 Modifying
- 1.4 Substrings and searching
- 1.5 `string_view` — the zero-copy slice
- 1.6 Conversions
- 1.7 Sorting, counting and comparing
- 1.8 The erase-remove idiom
- 1.9 Complexity summary — memorise this table

Part 2 · The mental model

- 2.1 The five questions
- 2.2 The one sentence for windows
- 2.3 The one sentence for frequency tables
- 2.4 Small alphabets change the complexity
- 2.5 Three costs that ruin otherwise-correct solutions
- 2.6 In-place means two pointers
- 2.7 The "string is the stack" realisation

Part 3 · Frequency counting in full

- 3.1 The 26-slot array
- 3.2 The three ways to compare two tables
- 3.3 The map version
- 3.4 The signature / canonical form

- 3.5 Counting with a bitmask
- 3.6 Choosing the container
- 3.7 The complete pattern in one place

Part 4 · Two pointers on strings

- 4.1 Shape 1 — converging from both ends
- 4.2 Shape 2 — the skip-one-mismatch palindrome
- 4.3 Shape 3 — read and write, both from the left
- 4.4 Shape 4 — one pointer per string

Part 5 · The sliding window, in full

- 5.1 The universal template
- 5.2 The three window flavours
- 5.3 Fixed-size window
- 5.4 Longest-valid window
- 5.5 The "at most K distinct" window
- 5.6 Longest Repeating Character Replacement — LC 424
- 5.7 Minimum window — the shrink-while-valid form
- 5.8 Recognising a window problem

Part 6 · Palindromes, in full

- 6.1 Checking one — two pointers, $O(n)$ / $O(1)$
- 6.2 Finding them — expand around centre
- 6.3 The DP table — when you need every range
- 6.4 Longest Palindromic Subsequence — a different problem
- 6.5 Manacher's algorithm — the $O(n)$ answer
- 6.6 The transformation trick
- 6.7 Quick decision table

Part 7 · Parsing, tokenising and building

- 7.1 Splitting on whitespace
- 7.2 Splitting on a delimiter
- 7.3 Comparing version numbers — LC 165
- 7.4 atoi by hand — LC 8
- 7.5 Roman numerals both ways — LC 13, LC 12
- 7.6 Zigzag conversion — LC 6

7.7 Building output efficiently

7.8 Largest Number — LC 179, the comparator problem

Part 8 · Pattern matching — naive, KMP and rolling hash

8.1 The naive scan

8.2 The prefix function — the idea behind KMP

8.3 Computing the prefix function

8.4 KMP search

8.5 What the prefix function is really for

8.6 Rolling hash (Rabin-Karp)

8.7 Choosing

Part 9 · Two-string DP

9.1 The grid

9.2 Longest Common Subsequence — LC 1143

9.3 Edit Distance — LC 72

9.4 Interleaving String — LC 97

9.5 Wildcard Matching — LC 44

9.6 Regular Expression Matching — LC 10

9.7 Space optimisation

9.8 Recognising the grid

Part 10 · The 14 patterns

How to choose, in order

The pairs that get confused

Part 11 · The six-step approach

Step 1 · Read for the three keywords

Step 2 · Say the pattern name before writing anything

Step 3 · Work one small example by hand

Step 4 · Answer the four window questions (or the DP equivalent)

Step 5 · Write it, then run the edge cases

Step 6 · State the complexity with the alphabet factor visible

The three sentences that earn credit

Part 12 · The 18-bug catalogue

Bug 1 · An uninitialised frequency array

Bug 2 · Unsigned underflow on size()

Bug 3 · Quadratic string building

Bug 4 · Not erasing zero-count keys

Bug 5 · A count signature with no separator

Bug 6 · Dropping the bound in a skip loop

Bug 7 · Single-digit run length in compression

Bug 8 · Forgetting the final carry

Bug 9 · Recording the window in the wrong place

Bug 10 · Moving the left pointer backwards in the jump variant

Bug 11 · The expand-around-centre overshoot

Bug 12 · Filling a palindrome DP table in the wrong order

Bug 13 · A separator that can occur in the input

Bug 14 · Losing the last token when splitting manually

Bug 15 · Negative modulo in a rolling hash

Bug 16 · Index versus count in a DP grid

Bug 17 · Missing DP base cases

Bug 18 · $j-1$ instead of $j-2$ for regex $*$

The pre-submit checklist

Part 13 · Complexity cheat sheet

std::string operations

Algorithms

Space, honestly

Reading the constraints

Part 14 · The curated 38-problem list

Tier 1 · Fundamentals (13 problems)

Tier 2 · The core patterns (15 problems)

Tier 3 · Advanced (10 problems)

The Indian-placement classics

If you only have time for eight

Part 15 · A three-week study plan

Week 1 · Characters, counting and two pointers

Week 2 · The sliding window

Week 3 · Matching and DP

How to know you are ready

What to do after this topic

Quick-reference card

Part 0 · Strings from zero

Before any pattern, you need to be completely certain about what a string *is* in memory. Almost every string bug in an interview comes from being fuzzy here, not from the algorithm.

0.1 A character is a number

A `char` in C++ is an integer type. It occupies exactly **1 byte** and stores a small number. The letter `'a'` is not stored as a picture of an `a`; it is stored as the number **97**.

```
char c = 'a';
cout << c;           // a      - printed as a character
cout << (int)c;       // 97     - the number underneath
cout << c + 1;        // 98     - arithmetic promotes char to int!
cout << char(c+1);    // b      - cast back to see the letter
```

The mapping from number to symbol is **ASCII**. You only need to memorise four anchors:

CHARACTER	ASCII VALUE
<code>'0'</code>	48
<code>'A'</code>	65
<code>'a'</code>	97
space <code>' '</code>	32

Everything else follows, because letters and digits are **contiguous**: `'b'` is 98, `'c'` is 99, `'9'` is 57.

That contiguity is the single most useful fact about characters, because it gives you the index trick:

```
int idx = c - 'a';      // 'a' -> 0, 'b' -> 1, ... 'z' -> 25
char back = 'a' + idx;  // 0 -> 'a', 1 -> 'b', ...
int digit = c - '0';    // '7' -> 7
```

The index trick is the foundation of every frequency array in this guide. `cnt[c - 'a']++` works because subtracting `'a'` turns a character into a slot number between 0 and 25.

Two more one-liners you will use constantly:

```
char lower = c | 32;    // 'A' (65) -> 'a' (97); 'a' stays 'a'
char upper = c & ~32;   // 'a' -> 'A'; 'A' stays 'A'
bool same = (a | 32) == (b | 32); // case-insensitive compare
```

They work because uppercase and lowercase letters differ by exactly one bit — bit 5, which is the value 32. `'A'` is `0100_0001` and `'a'` is `0110_0001`. Interviewers like this; use `tolower` in real code and mention the bit trick as an aside.

0.2 The classification functions

`<cctype>` gives you the standard tests. Know them by name — writing `c >= 'a' && c <= 'z'` by hand is fine but slower to read.

FUNCTION	TRUE WHEN
<code>isalpha(c)</code>	a-z or A-Z
<code>isdigit(c)</code>	0-9
<code>isalnum(c)</code>	letter or digit
<code>isspace(c)</code>	space, tab, newline, ...
<code>isupper(c)</code> / <code>islower(c)</code>	case tests
<code>tolower(c)</code> / <code>toupper(c)</code>	conversion (returns <code>int</code> , cast it)

Trap: these take an `int` and are only defined for values representable as `unsigned char` or `EOF`. With plain ASCII input this never bites, but the technically-correct call is `isalpha((unsigned char)c)`. Say that if an interviewer probes.

0.3 C strings versus `std::string`

C++ has two string types and you must be able to talk about both.

The **C string** is a raw array of `char` terminated by a `'\0'` byte:

```
char s[] = "cat";

index :   0   1   2   3
        +---+---+---+---+
        | c | a | t | \0 |
        +---+---+---+---+
        99  97 116   0
```

The `'\0'` is not decoration — it is the *only* way anything knows where the string ends. `strlen` walks forward counting bytes until it finds it, so `strlen` is **O(n)**, not O(1). The array above needs 4 bytes to hold 3 characters.

`std::string` is a class that owns a heap buffer plus a stored length:

```
std::string s = "cat";

s → [ pointer ] [ size = 3 ] [ capacity = 15 ]
    |
    ▼
    +---+---+---+---+
    | c | a | t |      (+ a '\0' kept for c_str())
    +---+---+---+---+
```

Because the length is stored, `s.size()` is **O(1)**. Because the buffer is owned, the string grows, copies and frees itself.

	C STRING CHAR[]	STD::STRING
Length	<code>strlen</code> , O(n)	<code>.size()</code> , O(1)
Grows	never	yes, automatically
Copy	<code>strcpy</code> , manual	<code>=</code> , deep copy
Compare	<code>strcmp</code>	<code>==</code> , <code><</code>
Concatenate	<code>strcat</code> , manual buffer	<code>+</code> , <code>+=</code>
Memory	yours to manage	managed

In interviews, always use `std::string` unless asked otherwise. Mention C strings only to show you know what is underneath.

0.4 Small String Optimisation

Modern `std::string` implementations store short strings (typically up to 15 characters on 64-bit libstdc++) **inside the string object itself**, with no heap allocation at all. That is why building tiny strings in a loop is cheaper than you would expect, and why `sizeof(std::string)` is 32 bytes rather than 8.

You will not be asked to implement it. You *may* be asked "is creating a small string expensive?" — the answer is "usually not, because of SSO; it becomes a heap allocation past about 15 characters."

0.5 Strings are mutable, and that matters

```
string s = "cat";
s[0] = 'b';           // legal – s is now "bat"
```

This is the difference from Java and Python, where strings are immutable and every modification allocates. In C++ you can rewrite a string **in place**, which is what makes the read/write two-pointer pattern (Part 4.3) possible in O(1) extra space.

0.6 Reading input without losing your mind

Three ways to read, and the trap that catches everyone:

```
string w; cin >> w;           // one whitespace-delimited word
string l; getline(cin, l);    // one whole line, newline consumed and discarded
```

The classic bug: `cin >> n;` followed by `getline(cin, line);` reads an **empty line**. `>>` leaves the trailing newline in the buffer and `getline` immediately finds it. Fix it with `cin.ignore();` — or `cin >> ws;` — between the two.

For splitting a line into tokens, `stringstream` is the tool:

```
string line = "the quick brown fox";
stringstream ss(line);
string word;
vector<string> words;
while (ss >> word) words.push_back(word); // 4 words, whitespace handled for you
```

Splitting on a specific delimiter uses the three-argument `getline`:

```
stringstream ss("1.12.0.3");
string part;
while (getline(ss, part, '.')) { /* "1", "12", "0", "3" */ }
```

Both of these show up constantly in parsing problems (LC 151, LC 165, LC 68).

Part 1 · The `std::string` toolkit

This is the API you are expected to know cold. Everything here is $O(1)$ or $O(n)$ as marked, and knowing *which* is how you avoid writing an accidental $O(n^2)$ solution.

1.1 Creating and sizing

```
string a;                // ""
string b = "hello";
string c(5, 'x');        // "xxxxx"      <- n copies of a char
string d(b);             // copy
string e(b.begin()+1, b.end()-1); // "ell"  <- iterator range
string f = to_string(42); // "42"

a.size();    a.length(); // identical, O(1), returns size_t
a.empty();   // O(1)
a.clear();   // size becomes 0
a.reserve(1000); // pre-allocate capacity, avoids re-allocation
```

`size()` returns an unsigned type. `s.size() - 1` when `s` is empty is not `-1`; it is 18446744073709551615. Every loop of the form `for (int i = 0; i < s.size() - 1; i++)` is a live bug on an empty string. Either cast — `(int)s.size()` — or store `int n = s.size();` once at the top. **Do this in every solution.**

1.2 Accessing characters

```
s[i]        // no bounds check – fastest, what you use in algorithms
s.at(i)     // throws std::out_of_range – safe, slower
s.front()   // s[0] (UB if empty)
s.back()    // s[size()-1] (UB if empty)
```

1.3 Modifying

```
s += 'c';           // append one char      – amortised O(1)
s += "text";        // append a string      – O(len added)
s.push_back('c');    // same as += 'c'
s.pop_back();        // remove last char     – O(1)
s.insert(3, "xy");    // insert at index 3   – O(n)
s.erase(2, 4);       // remove 4 chars from index 2 – O(n)
s.erase(s.begin()+2); // remove one char
s.replace(1, 3, "zz"); // replace 3 chars at index 1 with "zz"
reverse(s.begin(), s.end()); // <algorithm>, O(n), in place
```

Why `+=` is the right way to build a string. `s += c` is amortised $O(1)$: the buffer doubles when it fills, so n appends cost $O(n)$ total, not $O(n^2)$. This is the same amortised argument as `vector::push_back`.

The single biggest string-performance mistake: `s = s + c` inside a loop. That builds a whole new string every iteration — $O(n^2)$. `s += c` reuses the buffer — $O(n)$. They look almost identical and differ by a factor of n .

1.4 Substrings and searching

```
s.substr(pos)           // from pos to the end      - O(n), ALLOCATES
s.substr(pos, len)      // len characters from pos  - O(len), ALLOCATES
s.find("ab")            // first index, or string::npos
s.find('a', 5)          // search starting at index 5
s.rfind("ab")           // last occurrence
s.find_first_of("aeiou") // first index of ANY of these chars
s.find_first_not_of(" \t") // first index that is NOT one of these
s.compare(t)            // <0, 0, >0  (== and < also work directly)
```

The `npos` check is a fixed idiom:

```
size_t p = s.find("ab");
if (p != string::npos) { /* found at p */ }
```

substr allocates. It copies the characters into a brand-new string. Calling `substr` inside a loop over every index is $O(n^2)$ memory traffic and is the reason many "obvious" solutions time out. When you only need to *look* at a slice, use two indices, or `string_view`.

1.5 `string_view` — the zero-copy slice

`std::string_view` (C++17) is a pointer plus a length that borrows characters without copying. It has `size`, `substr`, `find`, `==`, iteration — everything except ownership and mutation.

```
#include <string_view>
string_view sv = s;
string_view slice = sv.substr(2, 5); // O(1), no allocation
```

Use it for read-only function parameters (`bool isPal(string_view s)`) and for slicing in hot loops. **Never** return a `string_view` into a temporary — the moment the underlying string dies, the view dangles.

Mentioning `string_view` in an interview is a cheap, genuine signal of modern C++ competence.

1.6 Conversions

```
int    x = stoi(s);      long long y = stoll(s);    double d = stod(s);
string s = to_string(x);
```

`stoi` skips leading whitespace, accepts an optional sign, stops at the first non-digit, and **throws** `std::invalid_argument` (nothing parseable) or `std::out_of_range` (too big). LC 8 (`atoi`) exists precisely to make you write that logic by hand, so know the four behaviours.

1.7 Sorting, counting and comparing

```
sort(s.begin(), s.end());           // "cba" -> "abc"    O(n log n)
count(s.begin(), s.end(), 'a');     // occurrences of 'a'
bool eq = (s == t);                 // O(n), compares sizes first
bool lt = (s < t);                   // lexicographic
string t = s; sort(t.begin(), t.end()); // the canonical "signature"
```

Sorting a string is how you build an **anagram signature** (Part 3.4). It is $O(n \log n)$ and the alternative — a 26-slot count — is $O(n)$. Know both, and know when the count version wins.

1.8 The erase-remove idiom

To delete every occurrence of a character:

```
s.erase(remove(s.begin(), s.end(), ' '), s.end()); // strip all spaces
```

`remove` does not shrink the container; it shifts the survivors forward and returns the new logical end. `erase` then truncates. One line, $O(n)$. In C++20 the free function `std::erase(s, ' ')` does both.

1.9 Complexity summary — memorise this table

OPERATION	COST	ALLOCATES?
<code>s[i]</code> , <code>size()</code> , <code>back()</code>	$O(1)$	no
<code>s += c</code>	amortised $O(1)$	occasionally
<code>s = s + c</code>	$O(n)$	every time
<code>s.substr(i, k)</code>	$O(k)$	yes
<code>s.find(t)</code>	$O(n \cdot m)$ worst case	no
<code>s.insert</code> / <code>s.erase</code> mid-string	$O(n)$	no
<code>sort(s)</code>	$O(n \log n)$	no
<code>s == t</code>	$O(n)$, $O(1)$ if sizes differ	no
<code>reverse(s)</code>	$O(n)$	no

Part 2 · The mental model

Every string problem in an interview is one of five questions in disguise. Learning to hear which one is being asked is worth more than learning any individual algorithm.

2.1 The five questions

THE QUESTION BEING ASKED	WHAT IT ALWAYS BECOMES
"Do these two things contain the same characters?"	a frequency table
"Find the best contiguous piece satisfying X"	a sliding window
"Compare a character to its mirror"	two pointers from the ends
"Does this string appear inside that one?"	pattern matching (naive → KMP → hash)
"Turn A into B" / "match A against B"	a 2-D DP grid

When you read a problem, your first job is not to start coding. It is to decide which row of that table you are in. Ninety percent of string problems are rows 1, 2 and 3.

The strongest signal is the word **contiguous**:

- **substring** / **subarray** = contiguous → sliding window or two pointers.
- **subsequence** = order preserved, gaps allowed → DP, or a greedy scan for the "is subsequence" variant.

Confusing those two words has broken more interviews than any algorithm ever has. `"ace"` is a subsequence of `"abcde"` but not a substring.

2.2 The one sentence for windows

A sliding window answers "what is the best contiguous piece?" by never moving the left pointer backwards.

That "never backwards" is the entire reason the window is $O(n)$ even though it contains a nested `while`. Each index enters the window once and leaves once, so there are at most $2n$ pointer moves in total. Say that sentence out loud when an interviewer asks why a doubly-nested loop is linear — it is the same amortised argument as the monotonic stack.

2.3 The one sentence for frequency tables

If the alphabet is small and fixed, use an array. If it is not, use a hash map.

`int cnt[26]` for lowercase letters, `int cnt[128]` for all of ASCII, `int cnt[256]` for arbitrary bytes, `unordered_map<char,int>` when you genuinely do not know. The array is 5–10× faster and has no hashing overhead. Interviewers notice when you reach for a map to count 26 lowercase letters.

2.4 Small alphabets change the complexity

This is the subtlety that separates a good answer from a great one. Many string algorithms have a factor of Σ , the alphabet size, hiding in them:

- Comparing two 26-slot frequency tables is $O(26) = O(1)$, not $O(n)$. That is what makes LC 438 (Find All Anagrams) $O(n)$ rather than $O(n \cdot 26)$ — and even $O(n \cdot 26)$ is $O(n)$.
- "At most K distinct characters" windows carry a map with at most Σ entries, so map operations are $O(1)$ in practice.
- A trie over lowercase letters branches 26 ways per node.

When you state a complexity, say " **$O(n)$ with a factor of 26 for the alphabet, which is constant**" rather than quietly dropping it. That is the sentence that shows you know where the cost went.

2.5 Three costs that ruin otherwise-correct solutions

1. `substr` in a loop. Generating every substring with `substr` is $O(n^3)$ total — $O(n^2)$ substrings, $O(n)$ to copy each. If you find yourself building substrings just to test them, you almost certainly want a window or two indices.

2. `s.find(t)` in a loop. Each call is $O(n \cdot m)$. Calling it once per position is $O(n^2 \cdot m)$. Build a prefix function once instead (Part 8).

3. String concatenation with `+`. Covered in Part 1.3 and worth repeating because it is invisible in a code review: `res = res + c` is quadratic, `res += c` is linear.

2.6 In-place means two pointers

If a problem says " **$O(1)$ extra space**" or "**modify in place**" and the input is a `string&` or a `vector<char>&`, the answer is almost always a pair of indices:

- one at each end, walking towards each other (reverse, palindrome);
- both starting at the left, one reading and one writing (compression, deduplication, removal).

The read/write pair is the more valuable of the two, because it generalises: *read* visits every character, *write* only advances when a character is kept, and at the end `write` is the new length. LC 443 (String Compression) is the canonical drill.

2.7 The "string is the stack" realisation

A string you are building up is already a stack: `push_back` pushes, `back()` peeks, `pop_back()` pops. You almost never need an actual `std::stack<char>` for a string problem.

```
string out;
for (char c : s) {
    if (!out.empty() && out.back() == c) out.pop_back(); // cancel
    else out.push_back(c);
}
```

That eight-line shape solves the whole "remove adjacent duplicates / backspace / collapse" family (LC 1047, LC 844, LC 1209) and leaves the answer already assembled — no reversing at the end, no transfer from a stack into a string.

Part 3 · Frequency counting in full

The most-used tool in string interviews. It looks trivial and has four distinct variants, each worth knowing by name.

3.1 The 26-slot array

```
int cnt[26] = {0}; // MUST be initialised - {0} zeroes all 26
for (char c : s) cnt[c - 'a']++;
```

`int cnt[26];` without the `= {0}` is filled with garbage. This is bug #1 in the catalogue and it is silent: the code compiles, runs, and gives a different wrong answer on every machine.

For all of ASCII use `int cnt[128] = {0};` and index with `cnt[c]` directly — no subtraction. Use 128 whenever the input can contain spaces, digits, punctuation or mixed case. Use `cnt[256]` and `(unsigned char)c` if bytes could be negative.

3.2 The three ways to compare two tables

Method A — count up, count down, check all zero. One table, one pass each way.

```
bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;
    int cnt[26] = {0};
    for (char c : s) cnt[c - 'a']++;
    for (char c : t) if (--cnt[c - 'a'] < 0) return false; // early exit
    return true;
}
```

The `< 0` test is the elegant part: if `t` ever has more of a character than `s` did, you know immediately without a final scan. And because the sizes are equal, no count can be left positive if none went negative.

Method B — two tables, compare with `memcmp`. Needed when you must compare repeatedly, as in a sliding window.

```
int a[26] = {0}, b[26] = {0};
// ... fill ...
if (memcmp(a, b, sizeof(a)) == 0) { /* identical */ }
```

`memcmp` on a 26-int array is 104 bytes — a handful of machine instructions.

Method C — a single "matches" counter. The advanced version, used by LC 567 and LC 438 to get true $O(n)$ with no 26-element comparison at all. You maintain a count of how many of the 26 slots currently agree, and update it by ± 1 as characters enter and leave.

```
if (++window[c] == need[c]) matches++;
else if (window[c] == need[c]+1) matches--; // just went one over
```

Get Method A and B fluent first; bring Method C out when the interviewer asks "can you avoid the 26-comparison?"

3.3 The map version

```
unordered_map<char,int> cnt;
for (char c : s) cnt[c]++;
```

`cnt[c]++` on a missing key **inserts it with value 0 first**, then increments — that is how `operator[]` on a map is defined. Convenient, but it means a mere *lookup* with `[]` also inserts. Use `.count(c)` or `.find(c)` when you only want to test membership, otherwise your map silently grows.

When you decrement to zero inside a window, **erase the key**:

```
if (--cnt[c] == 0) cnt.erase(c);
```

Otherwise `cnt.size()` — which you are probably using as "number of distinct characters" — counts keys whose value is zero and your window logic breaks. This is bug #4 and it is extremely common in "at most K distinct" problems.

3.4 The signature / canonical form

To group things that are equal up to rearrangement, map each one to a **canonical form** — a value that is identical for every member of the group.

```
string signature(const string& s) {
    string t = s;
    sort(t.begin(), t.end());    // "eat","tea","ate" -> "aet"
    return t;
}
```

Sorting is $O(k \log k)$ per word. The alternative signature is the count vector rendered as a string, which is $O(k)$:

```
string signature(const string& s) {
    int c[26] = {0};
    for (char ch : s) c[ch - 'a']++;
    string key;
    for (int i = 0; i < 26; i++) { key += '#'; key += to_string(c[i]); }
    return key;
    // "#1#0#0#0#1#...#1#..."
}
```

The `'#'` separator is essential: without it, counts of 1 and 12 versus 11 and 2 produce the same digits. This is bug #5.

Which to use? Sorting is shorter and fine when words are short (the usual case). The count key wins when words are long. **Say both in the interview**, then write the sort.

3.5 Counting with a bitmask

When you only care *whether* a character is present, not how many times, 26 bits fit in one `int`:

```
int mask = 0;
for (char c : s) mask |= 1 << (c - 'a');

bool hasA      = mask & (1 << 0);
bool disjoint  = (m1 & m2) == 0;      // no shared letter – LC 318
int  distinct  = __builtin_popcount(mask);
bool allUnique = __builtin_popcount(mask) == (int)s.size();
```

This turns "do these two words share a letter?" from an $O(26)$ loop into a single AND. LC 318 (Maximum Product of Word Lengths) is the standard use.

3.6 Choosing the container

SITUATION	USE
lowercase only, fixed alphabet	<code>int cnt[26]</code>
any ASCII, mixed case, punctuation	<code>int cnt[128]</code>
arbitrary bytes	<code>int cnt[256]</code> with <code>(unsigned char)c</code>
unknown / Unicode / word keys	<code>unordered_map<...,int></code>
presence only, ≤ 64 symbols	bitmask in <code>int</code> / <code>long long</code>
need the <i>smallest</i> or <i>largest</i> key	<code>map</code> (ordered) — $O(\log n)$

3.7 The complete pattern in one place

```
// "does s contain all characters of t, with multiplicity?" –  $O(n + m)$ 
bool canBuild(const string& s, const string& t) {
    int cnt[26] = {0};
    for (char c : s) cnt[c - 'a']++;
    for (char c : t) if (--cnt[c - 'a'] < 0) return false;
    return true;
}
```

That function, unchanged, is the answer to LC 383 (Ransom Note). With an equal-size guard in front, it is LC 242 (Valid Anagram). Slid across a string, it is LC 438 and LC 567. One idea, four problems.

Part 4 · Two pointers on strings

Three distinct shapes. Learn them as three separate templates, because mixing them up is where the bugs come from.

4.1 Shape 1 — converging from both ends

```
int l = 0, r = (int)s.size() - 1;
while (l < r) {
    // compare / swap s[l] and s[r]
    l++; r--;
}
```

Used for: reversing, palindrome checking, two-sum on a sorted string, the "reverse the vowels" family.

Why `l < r` and not `l <= r`? At `l == r` you are looking at the same character, which is trivially equal to itself and which you must not swap with itself. `<` handles both odd and even lengths correctly with no special case.

The filtering variant (LC 125, Valid Palindrome) — skip characters you do not care about, *inside* the loop, each skip guarded by `l < r`:

```
bool isPalindrome(string s) {
    int l = 0, r = (int)s.size() - 1;
    while (l < r) {
        while (l < r && !isalnum((unsigned char)s[l])) l++;
        while (l < r && !isalnum((unsigned char)s[r])) r--;
        if (tolower((unsigned char)s[l]) != tolower((unsigned char)s[r])) return false;
        l++; r--;
    }
    return true;
}
```

Both inner `while`s need `l < r` in the condition. Drop it and a string of pure punctuation walks `l` off the end of the string. This is bug #6, and it is the most common single mistake on LC 125.

4.2 Shape 2 — the skip-one-mismatch palindrome

LC 680 asks: can this be a palindrome after deleting **at most one** character? The insight is that you do not need to try every deletion. Walk inwards normally; the *first* mismatch is the only place a deletion could ever help, and there are exactly two candidates — drop the left character or drop the right one.

```

bool isPalRange(const string& s, int l, int r) {
    while (l < r) { if (s[l++] != s[r--]) return false; }
    return true;
}
bool validPalindrome(string s) {
    int l = 0, r = (int)s.size() - 1;
    while (l < r) {
        if (s[l] != s[r])
            return isPalRange(s, l + 1, r) || isPalRange(s, l, r - 1);
        l++; r--;
    }
    return true; // already a palindrome
}

```

$O(n)$, not $O(n^2)$, because the helper runs at most twice. This "the first mismatch is the only decision point" argument is the whole answer, and stating it clearly matters more than the code.

4.3 Shape 3 — read and write, both from the left

The in-place workhorse. One pointer reads every character; the other only advances when a character survives.

```

int write = 0;
for (int read = 0; read < n; read++) {
    if (keep(s[read])) s[write++] = s[read];
}
s.resize(write); // or: return write;

```

`write` is never ahead of `read`, so you never overwrite a character you have not read yet. That invariant is what makes in-place safe, and it is the sentence to say when asked why this works.

Worked example — LC 443, String Compression. Replace each run with its character followed by the run length (length omitted when 1), in place, $O(1)$ extra space.

```

int compress(vector<char>& ch) {
    int n = ch.size(), write = 0, read = 0;
    while (read < n) {
        char c = ch[read];
        int run = 0;
        while (read < n && ch[read] == c) { read++; run++; }
        ch[write++] = c;
        if (run > 1)
            for (char d : to_string(run)) ch[write++] = d; // 12 -> '1','2'
    }
    return write;
}

```

Two things to notice. First, the run length can be **multi-digit** — a run of 12 writes two characters. Writing `ch[write++] = run + '0'` breaks on any run of 10 or more, and that is bug #7. Second, `write` can never overtake `read`, because a run of length k is replaced by at most $1 + \text{digits}(k)$ characters, which is $\leq k$ for every $k \geq 1$.

4.4 Shape 4 — one pointer per string

Two strings, one index each, both advancing forward. This is how "is subsequence" and digit-by-digit arithmetic work.

```
// LC 392 — is s a subsequence of t? O(n + m)
bool isSubsequence(string s, string t) {
    int i = 0;
    for (char c : t) if (i < (int)s.size() && s[i] == c) i++;
    return i == (int)s.size();
}
```

Greedy is correct here: matching the earliest possible position in `t` never hurts, because any later match leaves strictly fewer characters for the rest of `s`. That exchange argument is the proof, and interviewers do ask for it.

The reverse-walk variant — string addition (LC 415) and multiplication walk both strings from the *end*, which is how you add numbers by hand:

```
string addStrings(string a, string b) {
    int i = a.size() - 1, j = b.size() - 1, carry = 0;
    string res;
    while (i >= 0 || j >= 0 || carry) {
        int x = i >= 0 ? a[i--] - '0' : 0;
        int y = j >= 0 ? b[j--] - '0' : 0;
        int s = x + y + carry;
        res += char('0' + s % 10);
        carry = s / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}
```

The `|| carry` in the loop condition is what handles `"99" + "1" = "100"`. Forgetting it gives `"00"` — bug #8. Building forwards and reversing once at the end is $O(n)$; inserting at the front each time would be $O(n^2)$.

Part 5 · The sliding window, in full

If you learn one thing from this guide, learn this part. The sliding window is the single highest-yield pattern in string interviews, and once you have the template it is the same seven lines every time.

5.1 The universal template

```
int l = 0;
for (int r = 0; r < n; r++) {
    add(s[r]); // 1. the new character enters
    while (windowIsInvalid()) { // 2. shrink until legal again
        remove(s[l]);
        l++;
    }
    best = max(best, r - l + 1); // 3. every window here is valid
}
```

Four questions turn any window problem into filled-in code:

1. **What does a character entering do?** (`add`)
2. **What makes the window illegal?** (`windowIsInvalid`)
3. **What does a character leaving undo?** (`remove`)
4. **Where do I record the answer** — inside the `while` or after it?

Answer those four out loud before typing. That is the whole method.

Why this is $O(n)$ despite the nested `while`: `l` only ever increases, and it can increase at most n times in the entire run. So the inner `while` executes at most n times across *all* iterations of the outer loop, not n times per iteration. Total pointer movement $\leq 2n$.

5.2 The three window flavours

FLAVOUR	SHAPE	EXAMPLE
Fixed size k	add <code>s[r]</code> ; if <code>r >= k</code> remove <code>s[r-k]</code> ; check when <code>r >= k-1</code>	LC 438, LC 567
Longest valid	<code>while (invalid) shrink;</code> record after the while	LC 3, LC 424, LC 340
Shortest valid	<code>while (valid) { record; shrink; }</code> record inside the while	LC 76, LC 209

The only structural difference between "longest" and "shortest" is whether the `while` condition is *invalid* or *valid*, and whether you record inside or outside it. Get that pairing wrong and the code looks right but answers the opposite question — bug #9.

5.3 Fixed-size window

No shrinking loop at all. The window is always exactly k wide once it fills, so one character enters and one leaves per step.

```
// LC 438 – all start indices of anagrams of p inside s
vector<int> findAnagrams(string s, string p) {
    int n = s.size(), k = p.size();
    vector<int> res;
    if (n < k) return res;
    int need[26] = {0}, win[26] = {0};
    for (char c : p) need[c - 'a']++;
    for (int r = 0; r < n; r++) {
        win[s[r] - 'a']++;
        if (r >= k) win[s[r - k] - 'a']--; // the character leaving
        if (r >= k - 1 && memcmp(need, win, sizeof(need)) == 0)
            res.push_back(r - k + 1);
    }
    return res;
}
```

Three index facts to get right, and they are the same three every time:

- the character leaving is `s[r - k]`, not `s[r - k + 1]`;
- the window is full from `r == k - 1` onwards;
- the window's left end is `r - k + 1`.

Complexity: $O(n \cdot 26) = O(n)$.

5.4 Longest-valid window

```
// LC 3 – longest substring without repeating characters
int lengthOfLongestSubstring(string s) {
    int cnt[128] = {0}, l = 0, best = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        cnt[(unsigned char)s[r]]++;
        while (cnt[(unsigned char)s[r]] > 1) { // the new char is the duplicate
            cnt[(unsigned char)s[l]]--;
            l++;
        }
        best = max(best, r - l + 1);
    }
    return best;
}
```

The `while` condition names *exactly* what went wrong: the character that just arrived now appears twice. Shrink until it does not. Notice that checking only `s[r]` is enough — the window was valid before `s[r]` arrived, so `s[r]` is the only possible offender.

There is a well-known **jump variant** that stores each character's last index and moves `l` in one step:

```
int last[128]; memset(last, -1, sizeof(last));
for (int r = 0; r < n; r++) {
    l = max(l, last[(unsigned char)s[r]] + 1); // never move l backwards!
    last[(unsigned char)s[r]] = r;
    best = max(best, r - l + 1);
}
```

The `max` is essential. Without it, an old occurrence far to the left drags `l` backwards and the window becomes invalid — bug #10. The jump version is not asymptotically faster; write the template version first and offer this one as a refinement.

5.5 The "at most K distinct" window

The workhorse behind a surprising number of problems.

```
int longestAtMostKDistinct(const string& s, int k) {
    unordered_map<char, int> cnt;
    int l = 0, best = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        cnt[s[r]]++;
        while ((int)cnt.size() > k) {
            if (--cnt[s[l]] == 0) cnt.erase(s[l]); // ERASE, or size() lies
            l++;
        }
        best = max(best, r - l + 1);
    }
    return best;
}
```

That `erase` is bug #4 from Part 3.3, and here you can see exactly why it matters: `cnt.size()` is the invalidity test.

The counting trick. "Exactly K distinct" has no direct window, because the window is not monotone in K. But:

$$\text{exactly}(K) = \text{atMost}(K) - \text{atMost}(K-1)$$

Two linear passes. The same subtraction handles "exactly K odd numbers", "exactly K vowels", and LC 992 / LC 1248. It is worth memorising as an identity.

5.6 Longest Repeating Character Replacement — LC 424

Replace at most `k` characters to make a window all-identical; find the longest such window.

```
int characterReplacement(string s, int k) {
    int cnt[128] = {0}, l = 0, maxFreq = 0, best = 0; // LC 424 is UPPERCASE
    for (int r = 0; r < (int)s.size(); r++) {
        maxFreq = max(maxFreq, ++cnt[(unsigned char)s[r]]);
        while ((r - l + 1) - maxFreq > k) { // replacements needed > k
            cnt[(unsigned char)s[l]]--;
            l++;
        }
        best = max(best, r - l + 1);
    }
    return best;
}
```

Note the 128-slot table: LC 424's alphabet is **uppercase**, so `c - 'a'` would index out of bounds. Indexing by the raw character sidesteps the whole question.

The validity test is the insight: a window of length `L` whose most frequent character appears `maxFreq` times needs `L - maxFreq` replacements. That is it.

The famous subtlety: `maxFreq` is **never decreased** when the window shrinks, so it can be stale — larger than the true maximum of the current window. This is deliberate and still correct. A stale `maxFreq` only makes the validity test more permissive, so the window never shrinks when it should not;

and `best` can only ever be updated by a window whose `maxFreq` was genuinely achieved. Being able to explain that is the whole point of the problem.

5.7 Minimum window — the shrink-while-valid form

LC 76 is the hardest window in common rotation, and it is the template with the `while` condition flipped.

```
string minWindow(string s, string t) {
    if (s.size() < t.size() || t.empty()) return "";
    int need[128] = {0};
    for (char c : t) need[(unsigned char)c]++;
    int missing = t.size(); // total characters still owed
    int l = 0, bestL = 0, bestLen = INT_MAX;
    for (int r = 0; r < (int)s.size(); r++) {
        if (need[(unsigned char)s[r]]-- > 0) missing--; // only useful chars count
        while (missing == 0) { // valid -> record, shrink
            if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestL = l; }
            if (++need[(unsigned char)s[l]] > 0) missing++;
            l++;
        }
    }
    return bestLen == INT_MAX ? "" : s.substr(bestL, bestLen);
}
```

Read `need[c]` as a **signed debt**: positive means "still owed", zero or negative means "satisfied, with surplus". `need[c]-- > 0` tests the value *before* decrementing, so `missing` drops only for a character that was genuinely still needed — a fourth 'a' when only three were required does not count. Symmetrically, `++need[l] > 0` raises `missing` only when removing that character actually breaks validity.

Store `bestL` and `bestLen` rather than calling `substr` inside the loop; one `substr` at the end instead of $O(n)$ of them.

5.8 Recognising a window problem

Reach for the window when you see **all** of:

- the answer is a **contiguous** substring or subarray;
- you want the longest / shortest / count of pieces satisfying a property;
- the property is **monotone** — if a window is valid, every sub-window of it is too (or symmetrically for invalid).

That monotonicity is what licenses "never move `l` backwards". Without it — for example with negative numbers in a "sum $\geq$ target" problem — the window is invalid and you need prefix sums plus a map instead. Knowing *when the window does not apply* is a senior-level answer.

Part 6 · Palindromes, in full

Palindromes are their own mini-topic. Four techniques, in increasing order of power, and you should be able to reach for the right one instantly.

6.1 Checking one — two pointers, $O(n)$ / $O(1)$

```
bool isPal(const string& s, int l, int r) {           // inclusive range
    while (l < r) if (s[l++] != s[r--]) return false;
    return true;
}
```

Take the range-based version, not the whole-string version. You will call it on slices constantly, and writing `isPal(s.substr(i, k))` instead costs an $O(k)$ allocation per call.

6.2 Finding them — expand around centre

The key counting fact: a string of length n has exactly $2n - 1$ centres. n centres sit on a character (odd-length palindromes) and $n - 1$ sit between characters (even-length). Every palindromic substring has exactly one centre, so enumerating centres enumerates palindromes without duplication.

```
"abba"    centres:  a  a|b  b  b|b  b  b|a  a
                  0   0.5 1   1.5 2   2.5 3       -> 7 = 2*4 - 1
```

```
// LC 647 — count all palindromic substrings.  $O(n^2)$  time,  $O(1)$  space.
int countSubstrings(string s) {
    int n = s.size(), total = 0;
    auto expand = [&](int l, int r) {
        while (l >= 0 && r < n && s[l] == s[r]) { total++; l--; r++; }
    };
    for (int c = 0; c < n; c++) {
        expand(c, c);           // odd centre
        expand(c, c + 1);       // even centre
    }
    return total;
}
```

The identical loop, tracking the longest instead of counting, is **LC 5 (Longest Palindromic Substring)**:

```
string longestPalindrome(string s) {
    int n = s.size(), bestL = 0, bestLen = n ? 1 : 0;
    auto expand = [&](int l, int r) {
        while (l >= 0 && r < n && s[l] == s[r]) { l--; r++; }
        int len = r - l - 1;           // note: l and r have overshoot by one
        if (len > bestLen) { bestLen = len; bestL = l + 1; }
    };
    for (int c = 0; c < n; c++) { expand(c, c); expand(c, c + 1); }
    return s.substr(bestL, bestLen);
}
```

The overshoot. When the `while` exits, `l` and `r` are each one step *past* the palindrome. So the length is `r - l - 1` and the start is `l + 1`. Writing `r - l + 1` here is bug #11 and it is the single most common palindrome error.

Expand-around-centre is $O(n^2)$ time and **$O(1)$ space**, which beats the $O(n^2)$ -space DP below on every metric except explainability. Make it your default answer for LC 5.

6.3 The DP table — when you need every range

```
// dp[i][j] = is s[i..j] a palindrome?
vector<vector<bool>> dp(n, vector<bool>(n, false));
for (int i = n - 1; i >= 0; i--) { // i descending
    dp[i][i] = true;
    for (int j = i + 1; j < n; j++)
        dp[i][j] = (s[i] == s[j]) && (j - i < 2 || dp[i + 1][j - 1]);
}
```

The iteration order is the whole trick. `dp[i][j]` depends on `dp[i+1][j-1]` — a *larger* `i` and a *smaller* `j`. So `i` must descend and `j` must ascend. Filling row-major top-to-bottom reads uninitialised cells. That is bug #12.

`j - i < 2` handles both base cases at once: length 1 (`j == i`, already set) and length 2 (`j == i+1`, where the inner range is empty and therefore vacuously a palindrome).

Use the table when a problem needs *repeated* "is `s[i..j]` a palindrome?" queries — palindrome partitioning (LC 131, LC 132) precomputes it and turns an exponential-looking search into a fast one.

6.4 Longest Palindromic Subsequence — a different problem

Subsequence, not substring — gaps allowed. Completely different recurrence, and the classic way this topic is made harder in an interview.

```
// LC 516. O(n^2) time, O(n^2) space.
int longestPalindromeSubseq(string s) {
    int n = s.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int i = n - 1; i >= 0; i--) {
        dp[i][i] = 1;
        for (int j = i + 1; j < n; j++)
            dp[i][j] = (s[i] == s[j]) ? dp[i+1][j-1] + 2
                                     : max(dp[i+1][j], dp[i][j-1]);
    }
    return dp[0][n-1];
}
```

There is an elegant alternative worth knowing: **$LPS(s) = LCS(s, reverse(s))$** . It is a one-liner given an LCS routine, and it is a satisfying thing to say out loud even if you then write the direct version.

6.5 Manacher's algorithm — the $O(n)$ answer

Manacher finds every palindromic radius in linear time by reusing the mirror image of previously computed radii around the current rightmost palindrome.

You are almost never expected to write it. Interviewers asking LC 5 want expand-around-centre. The correct move is: *"Expand-around-centre is $O(n^2)$ time and $O(1)$ space. Manacher's algorithm gets it to $O(n)$ by reusing mirrored radii inside the current rightmost palindrome; I can write it if you want, but it is rarely worth the bug risk in an interview."* That sentence gets full credit for knowing it exists.

Where Manacher genuinely earns its place is LC 214 (Shortest Palindrome) and LC 1312 — and for LC 214, the **KMP** solution in Part 8.5 is shorter and easier to defend.

6.6 The transformation trick

A string s is a palindrome **iff** `s == reverse(s)`. That turns several problems into a different problem you already know:

- longest palindromic *prefix* of $s \rightarrow$ longest prefix of `s` that is also a suffix of `reverse(s)` $\rightarrow$ **KMP prefix function** on `s + '#' + reverse(s)` (LC 214);
- longest palindromic *subsequence* $\rightarrow$ **LCS(s, reverse(s))** (LC 516);
- "is s a rotation of t " $\rightarrow$ `t.find(s) != npos` on `s + s`.

The `'#'` separator in the first one must be a character that cannot occur in the input, otherwise the match can straddle the join and report a prefix longer than `s` itself. That is bug #13.

6.7 Quick decision table

THE QUESTION	USE
Is this one string a palindrome?	two pointers, $O(n)/O(1)$
Longest palindromic substring	expand around centre
Count palindromic substrings	expand around centre
Palindrome after deleting ≤ 1 char	first-mismatch + two tries (Part 4.2)
Repeated "is <code>s[i..j]</code> a palindrome?"	DP table, filled <code>i</code> descending
Longest palindromic subsequence	interval DP, or LCS with the reverse
Shortest palindrome by prepending	KMP on <code>s + '#' + rev(s)</code>
Genuinely need $O(n)$ for substrings	Manacher (mention, rarely write)

Part 7 · Parsing, tokenising and building

The unglamorous half of string interviews, and the half most candidates under-prepare. These questions have no clever algorithm — they test whether you can enumerate cases without missing one.

7.1 Splitting on whitespace

`stringstream` handles arbitrary runs of whitespace, leading and trailing, for free:

```
vector<string> words;
stringstream ss(line);
string w;
while (ss >> w) words.push_back(w);
```

That single loop is the entire answer to **LC 151 (Reverse Words in a String)** once you reverse the vector:

```
string reverseWords(string s) {
    stringstream ss(s);
    string w, res;
    vector<string> v;
    while (ss >> w) v.push_back(w);
    for (int i = v.size() - 1; i >= 0; i--) {
        res += v[i];
        if (i) res += ' ';
    }
    return res;
}
```

The `if (i)` is what avoids a trailing space — a case the judge does check.

The **O(1)-space variant** is a favourite follow-up: reverse the whole string, then reverse each word in place, then squeeze out the extra spaces with the read/write pointer from Part 4.3. Know that it exists and that it is three passes.

7.2 Splitting on a delimiter

```
vector<string> split(const string& s, char d) {
    vector<string> out;
    stringstream ss(s);
    string part;
    while (getline(ss, part, d)) out.push_back(part);
    return out;
}
```

Note the difference from `>>`: `getline` with a delimiter **keeps empty fields**. `"a,,b"` split on `,` gives `{"a", "", "b"}`, which is usually what you want for CSV and version strings, and never what you want for words.

The manual version, when you are not allowed `<sstream>`:

```
vector<string> split(const string& s, char d) {
    vector<string> out;
    int start = 0;
    for (int i = 0; i <= (int)s.size(); i++)
        if (i == (int)s.size() || s[i] == d) { out.push_back(s.substr(start, i - start)); start = i
+ 1; }
    return out;
}
```

The `i <= size()` with the `i == size()` guard is how you flush the final field without duplicating the push. Writing `i < size()` silently drops the last token — bug #14.

7.3 Comparing version numbers — LC 165

The trap is that `"1.0"` and `"1.0.0"` are equal, and `"1.01"` equals `"1.1"`. So you cannot compare strings, and you cannot require equal numbers of parts. Parse both on the fly, treating a missing part as zero:

```
int compareVersion(string v1, string v2) {
    int i = 0, j = 0, n = v1.size(), m = v2.size();
    while (i < n || j < m) {
        long a = 0, b = 0;
        while (i < n && v1[i] != '.') a = a * 10 + (v1[i++] - '0');
        while (j < m && v2[j] != '.') b = b * 10 + (v2[j++] - '0');
        if (a != b) return a < b ? -1 : 1;
        i++; j++; // step over the '.' (harmless past the end)
    }
    return 0;
}
```

`while (i < n || j < m)` — **or**, not **and**. Once one string runs out, its part parses as 0 and the comparison continues correctly. That single character is the whole problem.

7.4 `atoi` by hand — LC 8

Pure case enumeration. Five steps, in this exact order:

```
int myAtoi(string s) {
    int i = 0, n = s.size();
    while (i < n && s[i] == ' ') i++; // 1. skip leading spaces
    int sign = 1;
    if (i < n && (s[i] == '+' || s[i] == '-')) // 2. at most one sign
        sign = (s[i++] == '-') ? -1 : 1;
    long res = 0; // 3. accumulate digits
    while (i < n && isdigit((unsigned char)s[i])) {
        res = res * 10 + (s[i++] - '0');
        if (sign == 1 && res > INT_MAX) return INT_MAX; // 4. clamp early
        if (sign == -1 && -res < INT_MIN) return INT_MIN;
    }
    return (int)(sign * res); // 5. stop at first non-digit
}
```

Clamp **inside** the loop, not after it. A 30-digit input overflows even a `long long` before you get a chance to check, so the test must happen on every digit. The strictly-correct version does the check without any wide type at all:

```
if (res > (INT_MAX - d) / 10) return sign == 1 ? INT_MAX : INT_MIN;
```

Say that version out loud; write whichever you are confident in. And remember the asymmetry:

`INT_MIN` is `-2147483648` while `INT_MAX` is `2147483647`, so the negative side allows one more.

7.5 Roman numerals both ways — LC 13, LC 12

Roman → integer. The rule is: a smaller value immediately before a larger one is subtracted.

```
int romanToInt(string s) {
    unordered_map<char, int> v{{'I', 1}, {'V', 5}, {'X', 10}, {'L', 50},
                               {'C', 100}, {'D', 500}, {'M', 1000}};

    int res = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        if (i + 1 < (int)s.size() && v[s[i]] < v[s[i+1]]) res -= v[s[i]];
        else res += v[s[i]];
    }
    return res;
}
```

Integer → Roman. Greedy over a table that already contains the six subtractive forms, so no special cases are needed at all:

```
string intToRoman(int num) {
    const int val[] = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
    const string sym[] = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"};
    string res;
    for (int i = 0; i < 13; i++)
        while (num >= val[i]) { num -= val[i]; res += sym[i]; }
    return res;
}
```

Putting `900/CM`, `400/CD`, `90/XC`, `40/XL`, `9/IX`, `4/IV` into the table *is* the trick. Greedy is then provably optimal because the table is constructed so that no smaller combination can reach a larger value.

7.6 Zigzag conversion — LC 6

The conversion looks geometric and is really just "which row does each character land in?" Track a row index and a direction that flips at the top and bottom:

```
string convert(string s, int numRows) {
    if (numRows == 1) return s; // the guard that matters
    vector<string> rows(min<int>(numRows, s.size()));
    int cur = 0, dir = -1;
    for (char c : s) {
        rows[cur] += c;
        if (cur == 0 || cur == numRows - 1) dir = -dir;
        cur += dir;
    }
    string res;
    for (auto& r : rows) res += r;
    return res;
}
```

`numRows == 1` must be special-cased: with one row the direction flips every step and `cur` walks out of range. Initialising `dir = -1` so that the first flip at row 0 makes it `+1` is the neat way to avoid an extra

branch.

7.7 Building output efficiently

```
string res;
res.reserve(estimatedSize);    // one allocation instead of log n
for (...) res += piece;        // never res = res + piece
```

When you must produce output in reverse order — digit arithmetic, stack unwinding — **append forwards and reverse once at the end**. `res.insert(res.begin(), c)` shifts the whole string on every call and turns an $O(n)$ routine into $O(n^2)$.

7.8 Largest Number — LC 179, the comparator problem

Given numbers as strings, concatenate them in the order that makes the largest number. The comparator is the answer and it is two characters of insight:

```
string largestNumber(vector<int>& nums) {
    vector<string> v;
    for (int x : nums) v.push_back(to_string(x));
    sort(v.begin(), v.end(),
        [](const string& a, const string& b) { return a + b > b + a; });
    if (v[0] == "0") return "0";           // all zeros
    string res;
    for (auto& s : v) res += s;
    return res;
}
```

Compare `a+b` against `b+a`, not `a` against `b` — `"9"` must beat `"34"`, and `"3"` must beat `"30"` (`"330" > "303"`). The `v[0] == "0"` guard turns `"000"` into `"0"`.

This comparator is a **strict weak ordering** (it is transitive because string concatenation comparison behaves like comparing the infinite repetitions), which is what makes `sort` legal — an interviewer may well ask why.

Part 8 · Pattern matching — naive, KMP and rolling hash

"Does `pat` occur inside `txt`?" Three answers, and you should be able to produce all three plus the reason to prefer each.

8.1 The naive scan

```
int strStr(string txt, string pat) {
    int n = txt.size(), m = pat.size();
    for (int i = 0; i + m <= n; i++) {
        int j = 0;
        while (j < m && txt[i + j] == pat[j]) j++;
        if (j == m) return i;
    }
    return m == 0 ? 0 : -1;
}
```

$O(n \cdot m)$ worst case — `"aaaaaaaaab"` searched for `"aaab"` re-compares the same prefix at every shift. In practice, on random text, it is close to $O(n)$, which is why it is a perfectly good answer to LC 28 and why `std::string::find` uses something similar.

Note `i + m <= n` in the loop condition rather than `i < n`: it stops early instead of relying on bounds checks inside.

8.2 The prefix function — the idea behind KMP

The waste in the naive scan is that after a mismatch we throw away everything we learned. KMP keeps it, using one precomputed array.

Definition. `pi[i]` = the length of the longest **proper** prefix of `pat[0..i]` that is also a suffix of `pat[0..i]`. "Proper" means it is not the whole thing.

```
pat  : a b a b a c a
index: 0 1 2 3 4 5 6
pi   : 0 0 1 2 3 0 1
```

Read `pi[4] = 3`: the prefix `"aba"` of `"ababa"` is also its suffix. So if we have matched 5 characters and the 6th mismatches, we do not restart — we already have 3 characters matched, and we retry from there.

8.3 Computing the prefix function

```
vector<int> prefixFunction(const string& p) {
    int m = p.size();
    vector<int> pi(m, 0);
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && p[i] != p[j]) j = pi[j - 1]; // fall back
        if (p[i] == p[j]) j++;
        pi[i] = j;
    }
    return pi;
}
```

Twelve lines that repay memorising outright. Three facts:

- `pi[0]` is always 0 — a single character has no proper prefix.
- The fallback is `j = pi[j-1]`, not `j--`. You jump to the next-longest border, not back by one.
- It is **O(m)**: `j` increases at most once per iteration and never goes below 0, so the total number of decreases across the whole loop is bounded by the total number of increases.

8.4 KMP search

```
int kmpFind(const string& txt, const string& pat) {
    if (pat.empty()) return 0;
    vector<int> pi = prefixFunction(pat);
    int j = 0;
    for (int i = 0; i < (int)txt.size(); i++) {
        while (j > 0 && txt[i] != pat[j]) j = pi[j - 1];
        if (txt[i] == pat[j]) j++;
        if (j == (int)pat.size()) return i - j + 1; // full match
        // for ALL matches: { record(i-j+1); j = pi[j-1]; }
    }
    return -1;
}
```

O(n + m) total, O(m) space. The search loop is the same shape as the build loop — that symmetry is how you remember it. The commented line is how you find every occurrence rather than the first: instead of returning, fall back as though the last character mismatched.

8.5 What the prefix function is *really* for

In interviews, KMP is asked for pattern search maybe a third of the time. The rest of the time it is the prefix function used directly as a tool for **self-similarity**:

Repeated Substring Pattern (LC 459). A string of length `n` is built from repetitions of a shorter block iff `n % (n - pi[n-1]) == 0` and `pi[n-1] > 0`. The quantity `n - pi[n-1]` is the smallest period.

```
bool repeatedSubstringPattern(string s) {
    int n = s.size();
    vector<int> pi = prefixFunction(s);
    int period = n - pi[n - 1];
    return pi[n - 1] > 0 && n % period == 0;
}
```

There is also the famous one-liner: `(s + s).substr(1, 2*n - 2).find(s) != npos`. It is cute and correct, and it is $O(n^2)$ with `find`. Mention it, write the KMP version.

Longest Happy Prefix (LC 1392) is literally `pi.back()` — the longest proper prefix that is also a suffix. The whole problem is the definition.

Shortest Palindrome (LC 214). Prepend the fewest characters to make `s` a palindrome. Using the transformation from Part 6.6:

```
string shortestPalindrome(string s) {
    if (s.empty()) return s;
    string rev(s.rbegin(), s.rend());
    string comb = s + '#' + rev;
    vector<int> pi = prefixFunction(comb);
    int k = pi.back();
    return rev.substr(0, s.size() - k) + s;
}
```

`pi.back()` is the longest prefix of `s` that is also a suffix of `reverse(s)` — which is exactly the longest **palindromic prefix** of `s`. Everything after it must be mirrored onto the front. The `'#'` is load-bearing: without it the border can run past the join and report a length greater than `s.size()`.

8.6 Rolling hash (Rabin-Karp)

Give every substring a number, so that comparing two substrings is comparing two integers.

```
// polynomial hash of s[i..i+k-1], rolled across the string
const long long B = 131, M = 1000000007LL;
long long h = 0, pow = 1;
for (int i = 0; i < k; i++) { h = (h * B + s[i]) % M; if (i) pow = pow * B % M; }
// pow == B^(k-1)
for (int i = k; i < (int)s.size(); i++) {
    h = (h - s[i-k] * pow % M + M) % M; // drop the leaving character
    h = (h * B + s[i]) % M;             // add the entering one
}
```

Each roll is $O(1)$, so scanning every window of size k is $O(n)$. Three rules:

1. **+ M before the final %**. Subtracting can go negative, and C++'s `%` keeps the sign of the dividend. `(-5) % 7` is `-5`, not `2`. This is bug #15 and it produces hashes that are almost right.
2. **Use long long throughout**. `h * B` with `h` near 10^9 and `B = 131` overflows a 32-bit int instantly.
3. **A hash match is not a match**. Verify with a real comparison, or use two independent moduli. For anti-hash adversarial tests, pick the base randomly at run time.

Where it wins: **LC 187 (Repeated DNA Sequences)** — fixed-width windows over a 4-letter alphabet, where 10 characters pack into 20 bits and the "hash" can be an exact bitmask with no collisions at all. And **LC 1044 (Longest Duplicate Substring)** — binary search the answer length, roll a hash of that width, check for duplicates in a set. That combination, binary search over length plus rolling hash, is the thing to recognise.

8.7 Choosing

SITUATION	USE
One search, short pattern, no constraints	<code>s.find(t)</code> or the naive scan
Guaranteed $O(n+m)$, adversarial input	KMP
Self-similarity: periods, borders, palindromic prefix	prefix function directly
Many patterns at once	Aho–Corasick (name it, do not write it)
Fixed-width windows, need set membership	rolling hash / bitmask
Binary search over substring length	rolling hash
Prefix queries, autocomplete	trie (Topic 14)

Part 9 · Two-string DP

When a problem takes **two** strings and asks you to transform, align or match them, the answer is a grid. The shape is always the same; only the recurrence changes.

9.1 The grid

`dp[i][j]` answers the question for **the first i characters of a and the first j characters of b** . Note the off-by-one on purpose: i is a *count*, not an index, so `dp[0][j]` is a real, meaningful base case (a is empty) and you never write `dp[-1]`.

```
      ""  b  b1 b2 ...
""  [0][0]
a1      dp[i-1][j-1] dp[i-1][j]
a2      dp[i][j-1]   dp[i][j]
```

Every recurrence below reads from at most three neighbours: diagonal, up, left. The character comparison is always `a[i-1] == b[j-1]` because of the count-versus-index shift. Writing `a[i] == b[j]` is bug #16 and it reads one character too far.

Size the table `(n+1) × (m+1)`.

9.2 Longest Common Subsequence — LC 1143

```
int longestCommonSubsequence(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i-1] == b[j-1]) ? dp[i-1][j-1] + 1
                                           : max(dp[i-1][j], dp[i][j-1]);
    return dp[n][m];
}
```

In words: if the last characters match, they can be paired — take the diagonal plus one. If they do not, one of them must be dropped — try both and take the better. Base row and column are zero, which `vector(..., 0)` already gives you.

LCS is the parent of a family. Longest common *substring* is the same grid with `dp[i][j] = a[i-1]==b[j-1] ? dp[i-1][j-1]+1 : 0` and the answer is the maximum cell, not the corner — because a substring must be contiguous, so a mismatch resets to zero. Minimum deletions to make two strings equal is `n + m - 2·LCS`. Shortest common supersequence is `n + m - LCS`. Longest palindromic subsequence is `LCS(s, reverse(s))`.

9.3 Edit Distance — LC 72

```
int minDistance(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) dp[i][0] = i;           // delete everything
    for (int j = 0; j <= m; j++) dp[0][j] = j;           // insert everything
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i-1] == b[j-1])
                ? dp[i-1][j-1]
                : 1 + min({dp[i-1][j-1], // replace
                           dp[i-1][j],   // delete from a
                           dp[i][j-1]}); // insert into a
    return dp[n][m];
}
```

Learn the three neighbours by what they *mean*, not by position:

NEIGHBOUR	OPERATION
<code>dp[i-1][j-1]</code>	replace <code>a[i-1]</code> with <code>b[j-1]</code>
<code>dp[i-1][j]</code>	delete <code>a[i-1]</code>
<code>dp[i][j-1]</code>	insert <code>b[j-1]</code>

The base cases are the part people forget. `dp[i][0] = i` because turning a length-*i* string into the empty string takes *i* deletions. Initialising the whole table to 0 and skipping those two loops gives a plausible-looking wrong answer — bug #17.

9.4 Interleaving String — LC 97

Can `c` be formed by interleaving `a` and `b`, preserving the order within each?

```
bool isInterleave(string a, string b, string c) {
    int n = a.size(), m = b.size();
    if (n + m != (int)c.size()) return false;           // the guard that matters
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= m; j++) {
            if (i && dp[i-1][j] && a[i-1] == c[i+j-1]) dp[i][j] = true;
            if (j && dp[i][j-1] && b[j-1] == c[i+j-1]) dp[i][j] = true;
        }
    return dp[n][m];
}
```

The index into `c` is `i + j - 1`: having consumed `i` from `a` and `j` from `b`, you are placing character number `i+j` of `c`, which lives at index `i+j-1`. That single expression is the whole problem, and it is why greedy fails — when both `a[i-1]` and `b[j-1]` equal `c[i+j-1]` there is no local way to choose.

9.5 Wildcard Matching — LC 44

`?` matches exactly one character, `*` matches any sequence including empty.

```

bool isMatch(string s, string p) {
    int n = s.size(), m = p.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;
    for (int j = 1; j <= m; j++) // empty s: only leading '*'s match
        dp[0][j] = dp[0][j-1] && p[j-1] == '*';
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (p[j-1] == '*')
                dp[i][j] = dp[i-1][j] || dp[i][j-1]; // consume a char | match empty
            else
                dp[i][j] = dp[i-1][j-1] && (p[j-1] == '?' || p[j-1] == s[i-1]);
        }
    return dp[n][m];
}

```

`dp[i-1][j]` is "`*` swallows `s[i-1]` and stays available"; `dp[i][j-1]` is "`*` matches nothing and we move past it". Two options, one `||`.

9.6 Regular Expression Matching — LC 10

`.` matches one character, `*` means **zero or more of the preceding element** — so `*` is never standalone and must always be read together with the character before it.

```

bool isMatch(string s, string p) {
    int n = s.size(), m = p.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;
    for (int j = 2; j <= m; j++) // "a*b*c*" can match empty
        if (p[j-1] == '*') dp[0][j] = dp[0][j-2];
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (p[j-1] == '*') {
                dp[i][j] = dp[i][j-2]; // zero occurrences
                if (p[j-2] == '.' || p[j-2] == s[i-1])
                    dp[i][j] = dp[i][j] || dp[i-1][j]; // one more occurrence
            } else {
                dp[i][j] = dp[i-1][j-1] && (p[j-1] == '.' || p[j-1] == s[i-1]);
            }
        }
    return dp[n][m];
}

```

The difference from wildcard, stated cleanly: in LC 44 `*` is independent and consumes text via `dp[i-1][j]`; in LC 10 `*` is glued to `p[j-2]` and "zero occurrences" skips **two** pattern characters via `dp[i][j-2]`. Writing `dp[i][j-1]` there is bug #18.

The base-row loop starts at `j = 2` because a `*` at index 0 would be malformed input.

9.7 Space optimisation

Every recurrence above reads only the current and previous rows, so two rows of size `m+1` suffice:

```

vector<int> prev(m + 1, 0), cur(m + 1, 0);
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) cur[j] = /* uses prev[j], prev[j-1], cur[j-1] */;
    swap(prev, cur);
}

```

$O(n \cdot m)$ time, **$O(m)$ space**. Offer it after the 2-D version works — it is the standard follow-up for LC 1143 and LC 72, and `swap` on vectors is $O(1)$ because it swaps the internal pointers.

A single rolling row is also possible if you save the diagonal in a temporary before overwriting it, but it is fiddly under pressure and rarely worth it.

9.8 Recognising the grid

Reach for two-string DP when:

- there are **two** strings and the question is transform / align / match / interleave;
- a greedy choice has a visible counterexample;
- the word is **subsequence**, not substring;
- the constraints are around $n, m \leq 1000$, which is exactly the size an $O(n \cdot m)$ table is designed for.

That last signal is worth taking seriously. $n \cdot m = 10^6$ cells is the classic DP budget; if the constraints are 10^5 you are not meant to build a grid at all.

Part 10 · The 14 patterns

This is the table to revise the night before an interview. Read the recognition signal, name the pattern, recall the core tool.

#	PATTERN	RECOGNITION SIGNAL	CORE TOOL	PROBLEMS
1	Two pointers, converging	palindrome, reverse, mirror, "from both ends"	<code>l = 0, r = n-1; while (l < r)</code>	LC 125, 344, 345, 680
2	Read / write pointer	"in place", "O(1) extra space", remove, compress	<code>if (keep) s[write++] = s[read]</code>	LC 443, 27, 26, 151
3	Frequency counting	anagram, permutation, "same characters", ransom note	<code>int cnt[26] / cnt[128]</code>	LC 242, 383, 387, 451
4	Canonical signature	group, bucket, "which of these are equivalent"	sorted string or <code>#</code> -joined count key	LC 49, 249
5	Fixed-size window	"of length k", "all anagrams of p", "substring of size m"	add <code>s[r]</code> , remove <code>s[r-k]</code>	LC 438, 567, 187, 30
6	Variable window — longest	"longest substring such that ..."	<code>while (invalid) shrink;</code> record after	LC 3, 424, 340, 904
7	Variable window — shortest	"minimum window", "smallest substring containing ..."	<code>while (valid) { record; shrink; }</code>	LC 76, 209, 1234
8	Expand around centre	palindromic substring, "count palindromes"	2n−1 centres, expand both ways	LC 5, 647, 1312
9	String as a stack	"remove adjacent", backspace, collapse, <code>k[...]</code>	<code>out.back()</code> , <code>out.pop_back()</code>	LC 1047, 844, 394, 1209
10	Parsing & tokenising	atoi, version, words, calculator, IP, path	<code>stringstream</code> , index walk, case list	LC 8, 151, 165, 68, 71
11	Character mapping	isomorphic, word pattern, "one-to-one"	two maps, both directions	LC 205, 290, 890
12	Prefix function (KMP)	"find the pattern", period, border, repeated block	<code>pi[i]</code> = longest proper border	LC 28, 214, 459, 1392
13	Rolling hash	fixed-width windows, "any duplicate substring", binary search on length	polynomial hash, <code>(x % M + M) % M</code>	LC 187, 1044, 1147
14	Two-string DP grid	two strings + transform / align / match / interleave	<code>(n+1) × (m+1)</code> table	LC 1143, 72, 10, 44, 97, 516

How to choose, in order

Run down this list and stop at the first match. It resolves nearly every string problem in under fifteen seconds.

1. **Two strings, "transform" or "match" or "subsequence"?** → pattern 14, the DP grid.
2. **The word "contiguous", "substring" or "subarray" plus longest/shortest/count?** → patterns 5–7, a window. Then decide fixed / longest / shortest.
3. **"Same characters" or "anagram" or "permutation"?** → pattern 3, and if it is sliding, pattern 5 too.
4. **"Palindrome"?** → pattern 1 to check one, pattern 8 to find them.
5. **"In place" or "O(1) space"?** → pattern 1 or 2.
6. **"Remove adjacent" or nested brackets?** → pattern 9.
7. **"Find pattern in text", or anything about periods and borders?** → pattern 12.
8. **Otherwise it is parsing** → pattern 10. Slow down and enumerate the cases.

The pairs that get confused

THESE LOOK ALIKE	THE DIFFERENCE
substring vs subsequence	contiguous vs gaps allowed — window vs DP
LC 3 vs LC 340	"no repeats" vs "at most k distinct" — same template, different invalidity test
LC 438 vs LC 567	list all start indices vs just "does one exist" — identical window
LC 44 vs LC 10	 standalone vs  glued to the previous character
LC 5 vs LC 516	palindromic substring (expand) vs subsequence (interval DP)
LC 242 vs LC 49	compare two vs group many — count table vs signature key

Part 11 · The six-step approach

Use this on every string problem until it is automatic. It is what stops you from writing code before you understand the question — the single most common reason good candidates fail.

Step 1 · Read for the three keywords

Underline them literally:

- **substring** / **subarray** → contiguous → window or two pointers.
- **subsequence** → gaps allowed → DP or a greedy scan.
- **in place** / **O(1) space** → two pointers, no extra containers.

Then check the alphabet: lowercase only, or full ASCII, or Unicode? That decides `cnt[26]` versus `cnt[128]` versus a map, and it is a question worth asking out loud.

Step 2 · Say the pattern name before writing anything

"This is a variable-size sliding window for the longest valid substring." Naming it out loud does two things: it lets the interviewer correct you in ten seconds rather than ten minutes, and it commits you to a template so you are not improvising structure while also improvising logic.

Step 3 · Work one small example by hand

Take a 5–6 character input and walk it on the whiteboard. For a window, write the table:

```
s = "abcabcbb", k = ?
r=0 window "a"      l=0 best=1
r=1 window "ab"     l=0 best=2
r=2 window "abc"    l=0 best=3
r=3 'a' repeats -> shrink to l=1, window "bca" best=3
```

Four rows is enough. This is where you discover that your invalidity test is the wrong way round, *before* it costs you twenty minutes.

Step 4 · Answer the four window questions (or the DP equivalent)

For a window:

1. What does a character entering do?
2. What makes the window illegal?
3. What does a character leaving undo?
4. Record the answer inside the shrink loop, or after it?

For a DP grid:

1. What does `dp[i][j]` mean, in one English sentence?
2. What are the base cases — row 0 and column 0?
3. What is the recurrence when the characters match? When they do not?

4. Which cell is the answer?

Write these four answers as comments, then fill the code in underneath them.

Step 5 · Write it, then run the edge cases

Six inputs, every time:

INPUT	WHAT IT CATCHES
<code>""</code>	unsigned underflow on <code>size() - 1</code> , empty-guard missing
<code>"a"</code>	off-by-one at the boundaries
<code>"aa"</code>	the even-length case
all identical, e.g. <code>"aaaa"</code>	window never shrinking, <code>maxFreq</code> logic
all distinct, e.g. <code>"abcd"</code>	window never growing
the pattern longer than the text	<code>n < k</code> guard, negative sizes

For pattern matching, add the empty pattern. For DP, add one empty string. For parsing, add leading and trailing spaces and a lone sign.

Step 6 · State the complexity with the alphabet factor visible

Not "it's $O(n)$ ". Say:

"Each index enters and leaves the window once, so $O(n)$ pointer moves. Comparing the two 26-slot tables is $O(26)$ per step, which is constant, so $O(n)$ overall. Space is $O(26) = O(1)$ — or $O(\min(n, \Sigma))$ if you want to be precise about the map version."

That is a complete answer. Then, unprompted, offer the follow-up you know is coming: the $O(1)$ -space version, the "what if it's Unicode" version, or the "what if the string is streaming and does not fit in memory" version.

The three sentences that earn credit

Memorise these; each one answers a question interviewers ask constantly.

1. **On why a nested `while` is linear:** *"The left pointer only moves forward, and it can move at most n times in total, so the inner loop runs at most n times across the entire outer loop — not n times per iteration."*
2. **On `substr`:** *"`substr` allocates and copies, so calling it inside a loop makes an $O(n)$ algorithm $O(n^2)$. I'll track two indices and build the result once at the end."*
3. **On small alphabets:** *"Comparing the frequency tables is $O(\Sigma)$, and Σ is 26 here, so it's constant — but I'd say $O(n \cdot \Sigma)$ rather than hide it, because for Unicode Σ is not small."*

Part 12 · The 18-bug catalogue

Every one of these has cost a real candidate a real offer. Read the list before any string interview; most of them are invisible on a quick re-read of your own code.

Bug 1 · An uninitialised frequency array

```
int cnt[26];           // x filled with garbage
int cnt[26] = {0};     // ✓
```

`= {0}` zeroes all 26 elements, not just the first — that is how aggregate initialisation works. Without it the code compiles, runs, and is wrong differently on every run. For `vector`, `vector<int> cnt(26, 0)` is already zeroed, and `vector<int> cnt(26)` is too.

Bug 2 · Unsigned underflow on `size()`

```
for (int i = 0; i < s.size() - 1; i++)    // x infinite loop when s is empty
int n = s.size();
for (int i = 0; i < n - 1; i++)           // ✓
```

`size()` returns `size_t`, which is unsigned. `0 - 1` becomes 18446744073709551615. **Store the size in an `int` once at the top of every function.** This single habit removes an entire class of bug.

Bug 3 · Quadratic string building

```
res = res + c;           // x new string every iteration - O(n²)
res += c;                // ✓ amortised O(1)
res.insert(res.begin(), c); // x also O(n²) - shifts everything
```

To build in reverse, append forwards and `reverse` once at the end.

Bug 4 · Not erasing zero-count keys

```
if (--cnt[s[l]] == 0) cnt.erase(s[l]); // ✓
```

If `cnt.size()` is your "number of distinct characters", a key sitting at zero inflates it and the window never shrinks far enough. Silent, and it only shows on certain inputs.

Bug 5 · A count signature with no separator

```
key += to_string(c[i]);           // x counts 1,12 and 11,2 collide
key += '#'; key += to_string(c[i]); // ✓
```

Bug 6 · Dropping the bound in a skip loop

```
while (!isalnum(s[l])) l++;           // x runs off the end on ".,.,,"
while (l < r && !isalnum(s[l])) l++; // ✓
```

Every inner skip loop needs the outer loop's bound repeated inside it.

Bug 7 · Single-digit run length in compression

```
ch[write++] = run + '0';           // x breaks at run >= 10
for (char d : to_string(run)) ch[write++] = d; // ✓
```

Bug 8 · Forgetting the final carry

```
while (i >= 0 || j >= 0)           // x "99" + "1" = "00"
while (i >= 0 || j >= 0 || carry) // ✓
```

Bug 9 · Recording the window in the wrong place

For **longest**: `while (invalid) shrink;` then record — the window is valid only after the loop.

For **shortest**: `while (valid) { record; shrink; }` — the window is valid only inside the loop.

Swap them and the code runs, looks right, and answers the other question.

Bug 10 · Moving the left pointer backwards in the jump variant

```
l = last[s[r]] + 1;           // x an old occurrence drags l back
l = max(l, last[s[r]] + 1);    // ✓
```

The window's whole correctness argument is "`l` never decreases".

Bug 11 · The expand-around-centre overshoot

```
while (l >= 0 && r < n && s[l] == s[r]) { l--; r++; }
int len = r - l - 1;           // ✓ both have overshoot by one
int start = l + 1;             // ✓
```

`r - l + 1` is right for an inclusive range but wrong here, because the loop exited by stepping one too far on each side.

Bug 12 · Filling a palindrome DP table in the wrong order

`dp[i][j]` depends on `dp[i+1][j-1]`, so `i` must descend and `j` must ascend. A plain row-major fill reads uninitialised cells and returns plausible nonsense.

Bug 13 · A separator that can occur in the input

```
string comb = s + '#' + rev;    // '#' must be impossible in s
```

If the separator appears in the data, the border can straddle the join. Pick a character outside the alphabet, and say so out loud.

Bug 14 · Losing the last token when splitting manually

```
for (int i = 0; i <= n; i++)  
    if (i == n || s[i] == d) { push(s.substr(start, i - start)); start = i + 1; }
```

The `i == n` arm is what flushes the final field. Looping `i < n` drops it.

Bug 15 · Negative modulo in a rolling hash

```
h = (h - s[i-k] * pow % M) % M;    // x can be negative  
h = ((h - s[i-k] * pow % M) % M + M) % M;    // ✓
```

C++'s `%` keeps the sign of the dividend: `(-5) % 7 == -5`. Also: use `long long` everywhere, and verify a hash hit with a real comparison.

Bug 16 · Index versus count in a DP grid

`dp[i][j]` covers the first `i` characters, so the character in question is `a[i-1]`, not `a[i]`. Every two-string DP recurrence compares `a[i-1] == b[j-1]`.

Bug 17 · Missing DP base cases

Edit distance needs `dp[i][0] = i` and `dp[0][j] = j`. Wildcard and regex need the base **row** built from leading `*`s. Zero-initialising the table and skipping these loops gives an answer that is wrong only on inputs where one string is a prefix of the other — which the sample cases rarely cover.

Bug 18 · `j-1` instead of `j-2` for regex `*`

In LC 10, `*` binds to the character before it, so "zero occurrences" skips **two** pattern characters: `dp[i][j] = dp[i][j-2]`. In LC 44, `*` stands alone and `dp[i][j-1]` is correct. The two problems look nearly identical and this is the line that differs.

The pre-submit checklist

Run these seven questions over your code before you say "I'm done":

1. Is every frequency array **initialised**?
2. Did I store `int n = s.size()` instead of using the unsigned value?
3. Am I using `+=` and never `s = s + c`?
4. Does every inner `while` repeat the outer bound?

5. For a window: is the record **inside** or **outside** the shrink loop, and does that match longest / shortest?
6. Did I test `" "`, `"a"`, `"aa"`, all-same and all-distinct?
7. Can I state the complexity with the alphabet factor visible?

Part 13 · Complexity cheat sheet

`std::string` operations

OPERATION	TIME	SPACE
<code>s[i]</code> , <code>size()</code> , <code>empty()</code> , <code>back()</code>	$O(1)$	—
<code>s += c</code>	amortised $O(1)$	occasional realloc
<code>s = s + c</code>	$O(n)$	new buffer each time
<code>s.substr(i, k)</code>	$O(k)$	$O(k)$, allocates
<code>s.find(t)</code>	$O(n \cdot m)$ worst	$O(1)$
<code>s.insert</code> / <code>s.erase</code> in the middle	$O(n)$	$O(1)$
<code>s == t</code>	$O(n)$, $O(1)$ if sizes differ	$O(1)$
<code>s < t</code>	$O(n)$	$O(1)$
<code>sort(s.begin(), s.end())</code>	$O(n \log n)$	$O(\log n)$
<code>reverse(s.begin(), s.end())</code>	$O(n)$	$O(1)$
<code>to_string(x)</code> / <code>stoi(s)</code>	$O(\text{digits})$	$O(\text{digits})$

Algorithms

ALGORITHM	TIME	SPACE	WHERE
Frequency count	$O(n + \Sigma)$	$O(\Sigma)$	Part 3
Anagram check	$O(n)$	$O(\Sigma)$	Part 3.2
Anagram check by sorting	$O(n \log n)$	$O(n)$	Part 3.4
Two pointers	$O(n)$	$O(1)$	Part 4
Sliding window	$O(n)$	$O(\Sigma)$	Part 5
Fixed window + table compare	$O(n \cdot \Sigma) \rightarrow O(n)$	$O(\Sigma)$	Part 5.3
Expand around centre	$O(n^2)$	$O(1)$	Part 6.2
Palindrome DP table	$O(n^2)$	$O(n^2)$	Part 6.3
Manacher	$O(n)$	$O(n)$	Part 6.5
Naive search	$O(n \cdot m)$	$O(1)$	Part 8.1
KMP (build + search)	$O(n + m)$	$O(m)$	Part 8.4
Rabin-Karp	$O(n + m)$ expected	$O(1)$	Part 8.6
Group anagrams	$O(n \cdot k \log k)$	$O(n \cdot k)$	LC 49
LCS / edit distance	$O(n \cdot m)$	$O(n \cdot m) \rightarrow O(m)$	Part 9
Longest palindromic subsequence	$O(n^2)$	$O(n^2) \rightarrow O(n)$	Part 6.4
Trie build / lookup	$O(\text{total}) / O(\text{len})$	$O(\text{total} \cdot \Sigma)$	Topic 14

Σ is the alphabet size: 26 for lowercase, 128 for ASCII.

Space, honestly

Say **$O(1)$ extra space** only when you truly allocate nothing that grows with n . A `cnt[26]` array is $O(1)$ because 26 is a constant — that is legitimate and worth saying explicitly. A `vector<string>` of the words is $O(n)$. The output itself is conventionally not counted, but say " $O(n)$ for the output, $O(1)$ auxiliary" rather than leaving it ambiguous.

Recursion counts: a recursive palindrome check is $O(n)$ stack space, not $O(1)$.

Reading the constraints

CONSTRAINT ON N	WHAT IS EXPECTED
$n \leq 20$	exponential is fine — backtracking, subsets
$n \leq 500$	$O(n^3)$
$n \leq 5\,000$	$O(n^2)$ — expand around centre, palindrome DP
$n, m \leq 1\,000$	$O(n \cdot m)$ — the two-string DP grid
$n \leq 10^5$	$O(n \log n)$ or $O(n)$ — window, KMP, hash
$n \leq 10^6$	$O(n)$ only, and watch the constant factor

This table is how you reverse-engineer the intended solution before writing a line. If $n \leq 5000$ the setter is *expecting* $O(n^2)$; do not burn twenty minutes hunting for a linear algorithm that is not there.

Part 14 · The curated 38-problem list

Every one of these is solved in full in [Strings-Solved-Questions.pdf](#), in this exact order. Attempt the problem on LeetCode first. Open the book only after you have genuinely struggled — and then still read the *Watch out* box, because it names the bug you probably almost made.

Tier 1 · Fundamentals (13 problems)

Build fluency with characters, counting and the basic two-pointer shapes. Do all thirteen before touching Tier 2.

#	PROBLEM	DIFFICULTY	PATTERN
1	LC 344 · Reverse String	Easy	two pointers, converging
2	LC 125 · Valid Palindrome	Easy	two pointers with filtering
3	LC 242 · Valid Anagram	Easy	frequency counting
4	LC 387 · First Unique Character	Easy	frequency counting, two passes
5	LC 383 · Ransom Note	Easy	frequency counting
6	LC 14 · Longest Common Prefix	Easy	vertical scan
7	LC 58 · Length of Last Word	Easy	reverse scan
8	LC 28 · Find the First Occurrence	Easy	naive pattern matching
9	LC 205 · Isomorphic Strings	Easy	character mapping, both ways
10	LC 392 · Is Subsequence	Easy	one pointer per string
11	LC 13 · Roman to Integer	Easy	parsing, look-ahead rule
12	LC 12 · Integer to Roman	Medium	greedy over a table
13	LC 415 · Add Strings	Easy	digit arithmetic from the end

Tier 2 · The core patterns (15 problems)

This is the tier that actually gets asked. LC 3, LC 5, LC 49, LC 76 and LC 438 are the five most-seen string questions in on-campus drives.

#	PROBLEM	DIFFICULTY	PATTERN
14	LC 3 · Longest Substring Without Repeating Characters	Medium	variable window, longest
15	LC 438 · Find All Anagrams in a String	Medium	fixed window + count table
16	LC 567 · Permutation in String	Medium	fixed window + count table
17	LC 424 · Longest Repeating Character Replacement	Medium	window with <code>len - maxFreq ≤ k</code>
18	LC 76 · Minimum Window Substring	Hard	shrink-while-valid window
19	LC 49 · Group Anagrams	Medium	canonical signature
20	LC 5 · Longest Palindromic Substring	Medium	expand around centre
21	LC 647 · Palindromic Substrings	Medium	expand around centre
22	LC 680 · Valid Palindrome II	Easy	first mismatch, two tries
23	LC 151 · Reverse Words in a String	Medium	tokenise, or three reversals
24	LC 443 · String Compression	Medium	read/write pointer
25	LC 8 · String to Integer (atoi)	Medium	parsing, case enumeration
26	LC 165 · Compare Version Numbers	Medium	parallel parsing
27	LC 6 · Zigzag Conversion	Medium	row simulation
28	LC 179 · Largest Number	Medium	custom comparator <code>a+b > b+a</code>

Tier 3 · Advanced (10 problems)

KMP, hashing and the DP grid. Do these once Tier 2 is comfortable — they are what separate a good on-campus result from a great one.

#	PROBLEM	DIFFICULTY	PATTERN
29	LC 214 · Shortest Palindrome	Hard	KMP on <code>s + '#' + rev(s)</code>
30	LC 459 · Repeated Substring Pattern	Easy	prefix function, period test
31	LC 1392 · Longest Happy Prefix	Hard	prefix function, directly
32	LC 187 · Repeated DNA Sequences	Medium	fixed window + rolling bitmask
33	LC 516 · Longest Palindromic Subsequence	Medium	interval DP
34	LC 1143 · Longest Common Subsequence	Medium	two-string DP grid
35	LC 72 · Edit Distance	Medium	two-string DP grid
36	LC 10 · Regular Expression Matching	Hard	DP, <code>*</code> binds to <code>p[j-2]</code>
37	LC 44 · Wildcard Matching	Hard	DP, <code>*</code> standalone
38	LC 97 · Interleaving String	Medium	DP, index <code>c[i+j-1]</code>

The Indian-placement classics

These appear in written rounds and on GeeksforGeeks far more often than on LeetCode, and several Indian service companies still ask them verbatim.

Written-round staples

- Reverse a string **without** any extra variable (XOR swap, or `swap`).
- Check two strings are rotations of each other — `(s + s).find(t) != npos`.
- Remove all duplicates from a string, preserving first-occurrence order.
- Print all permutations of a string (and the distinct-only variant).
- First non-repeating character **in a stream** — queue plus count array.
- Count and say / look-and-say sequence.
- Longest common prefix of an array of strings, all four standard approaches.
- String to integer and integer to string, by hand, with overflow handling.
- Check whether a string is a valid shuffle of two other strings.
- Recursively remove adjacent duplicates.
- Smallest window containing all distinct characters of the string itself.
- Convert a sentence to "camelCase" / toggle case / capitalise each word.

Aptitude-style favourites

- Count the vowels, consonants, digits and spaces in one pass.
- Check for a pangram; find the missing letters.
- Longest word in a sentence; word frequency counts.
- Check balanced parentheses (also Topic 04).
- Run-length encode and decode.

Company notes. TCS Digital and Infosys favour the parsing and counting problems above. Amazon, Microsoft and Adobe favour LC 3, 5, 76, 438 and the DP grid. Goldman Sachs and the quant firms like LC 8, LC 165 and LC 179 — the ones with hidden cases rather than hidden algorithms.

If you only have time for eight

LC 3, LC 5, LC 49, LC 76, LC 242, LC 438, LC 72, LC 28.

Those eight cover: the variable window, expand-around-centre, canonical signatures, the shrink-while-valid window, frequency counting, the fixed window, the DP grid, and pattern matching. Every other problem in this list is a variation on one of them.

Part 15 · A three-week study plan

Roughly 90 minutes a day. Adjust the pace, not the order — each week depends on the one before it.

Week 1 · Characters, counting and two pointers

DAY	READ	DO
1	Parts 0–1	<code>practice-01-basics.cpp</code> TODOs 1–3. Write <code>reverseString</code> , <code>isPalindrome</code> , <code>isAnagram</code> from a blank page.
2	Part 2	LC 344, LC 125, LC 242. Time yourself — each should take under ten minutes.
3	Part 3	LC 383, LC 387, LC 205. Do LC 205 with two maps and explain why one is not enough.
4	Part 4.1–4.2	LC 680, LC 14, LC 58.
5	Part 4.3–4.4	TODOs 4–5. Then LC 443 and LC 415 — both in place, no <code>substr</code> .
6	Part 7.5	LC 13, LC 12, LC 392.
7	—	Review. Redo LC 125 and LC 443 from scratch without looking. If either takes more than fifteen minutes, repeat Week 1 days 4–5.

At the end of Week 1 you should be able to write a frequency count and a converging two-pointer loop without thinking about the syntax at all.

Week 2 · The sliding window

This is the week that matters most. Do not move on until the template is automatic.

DAY	READ	DO
8	Part 5.1–5.3	TODO 6. Then LC 438 and LC 567 — notice they are the same code.
9	Part 5.4	LC 3, twice: the template version, then the jump version. Explain the <code>max</code> to yourself out loud.
10	Part 5.5–5.6	LC 424, LC 340 (or LC 904). Explain why the stale <code>maxFreq</code> is still correct.
11	Part 5.7	LC 76. Expect this to take an hour. Then re-derive <code>missing</code> from scratch the next morning.
12	Part 6.1–6.3	TODOs 7–8. Then LC 5 and LC 647.
13	Part 7.1–7.4	LC 151, LC 8, LC 165, LC 6.
14	Part 7.8 · Part 3.4	LC 49, LC 179. Review: write the window template from memory on paper.

At the end of Week 2 you should be able to write the seven-line window template from memory and fill in the four questions for a problem you have never seen.

Week 3 · Matching and DP

DAY	READ	DO
15	Part 8.1-8.4	TODO 9. Write <code>prefixFunction</code> from memory three times. Then LC 28 with KMP.
16	Part 8.5	LC 459, LC 1392. Both are the prefix function used directly.
17	Part 8.6	LC 187. Then read LC 1044 and understand the binary-search-plus-hash shape.
18	Part 9.1-9.3	TODO 10. Then LC 1143 and LC 72. Draw the grid by hand for a 4×4 example.
19	Part 9.4-9.6	LC 97, LC 44.
20	Part 9.6 · Part 6.4	LC 10, LC 516.
21	Parts 10-13	Full review. The 14 patterns, the bug catalogue, the complexity table. Redo LC 76 and LC 72 from scratch.

How to know you are ready

You are ready when you can, from a blank page and without reference:

- [] write the seven-line sliding-window template and name the four questions;
- [] write `prefixFunction` correctly, including the `j = pi[j-1]` fallback;
- [] write expand-around-centre with the correct `r - l - 1` length;
- [] write the LCS and edit-distance recurrences and their base cases;
- [] explain why the nested `while` in a window is $O(n)$;
- [] explain why `substr` in a loop is a performance bug;
- [] say what `dp[i][j]` means in one English sentence for any grid problem;
- [] name the pattern for a problem you have never seen, within fifteen seconds.

What to do after this topic

Strings feed directly into four later topics:

- **Topic 08 Hashing** — the frequency tables here are the entry point; the rolling hash is a preview.
- **Topic 14 Tries** — prefix matching, autocomplete, the word-search family.
- **Topic 11 Dynamic Programming** — the two-string grid in Part 9 *is* one of the standard DP shapes.
- **Topic 05 Recursion & Backtracking** — permutations of a string, palindrome partitioning, word break.

The sliding window itself reappears in Topic 01 Arrays, and the string-as-stack pattern in Part 2.7 is the same idea as Topic 04.

Quick-reference card

Tear this page out. It is everything you need in the ten minutes before an interview.

The window template

```
int l = 0;
for (int r = 0; r < n; r++) {
    add(s[r]);
    while (invalid()) { remove(s[l]); l++; }
    best = max(best, r - l + 1);
}
```

Longest → record **after** the while. Shortest → `while (valid) { record; shrink; }`.

Frequency count

```
int cnt[26] = {0}; // ALWAYS initialise
for (char c : s) cnt[c - 'a']++;
for (char c : t) if (--cnt[c - 'a'] < 0) return false;
```

Converging two pointers

```
int l = 0, r = n - 1;
while (l < r) { /* compare or swap */ l++; r--; }
```

Read / write, in place

```
int w = 0;
for (int rd = 0; rd < n; rd++) if (keep(s[rd])) s[w++] = s[rd];
```

Expand around centre — $2n - 1$ centres

```
while (l >= 0 && r < n && s[l] == s[r]) { l--; r++; }
int len = r - l - 1, start = l + 1; // both overshoot by one
```

Prefix function (KMP)

```
vector<int> pi(m, 0);
for (int i = 1; i < m; i++) {
    int j = pi[i-1];
    while (j > 0 && p[i] != p[j]) j = pi[j-1];
    if (p[i] == p[j]) j++;
    pi[i] = j;
}
```

Smallest period = `n - pi[n-1]`. Longest border = `pi[n-1]`.

Two-string DP

```

dp[i][j] // first i of a, first j of b -> compare a[i-1] and b[j-1]
LCS : match ? dp[i-1][j-1]+1 : max(dp[i-1][j], dp[i][j-1])
Edit : match ? dp[i-1][j-1] : 1 + min({dp[i-1][j-1], dp[i-1][j], dp[i][j-1]})
Base : dp[i][0] = i, dp[0][j] = j (edit distance only)

```

Character arithmetic

```

c - 'a' // 0..25 c - '0' // digit value
'a' + i // back to a char c | 32 // to lowercase

```

The seven checks before you submit

1. frequency array initialised · 2. `int n = s.size()` · 3. `+=` never `+`
2. inner `while` repeats the bound · 5. record inside vs outside matches longest vs shortest
3. tested `""`, `"a"`, `"aa"`, all-same, all-distinct · 7. complexity stated with Σ visible